

A Safety Profile Verifying Initialization

Document #: P3402R3
Date: 2025-05-16
Project: Programming Language C++
Audience: SG23, EWG
Reply-to: Marc-André Laverdière, Black Duck Software
<marc-andre.laverdiere@blackduck.com>
Christopher Lapkowski, Black Duck Software
<redacted@blackduck.com>
Charles-Henri Gros, Black Duck Software
<redacted@blackduck.com>

Contents

- 1 Abstract
- 2 Introduction
 - 2.1 Industry Demand
- 3 Design Objectives
 - 3.1 Main Objective
 - 3.2 Profile Quality Objectives
 - 3.3 Non-objectives
- 4 Definitions
- 5 Exemptions and Interactions with Non-Verified Code
- 6 Constraints on Verified Code
 - 6.1 General Constraints
 - 6.2 Verified Globals
 - 6.3 Verified Classes
 - 6.4 Verified Functions
 - 6.4.1 For All Verified Functions
 - 6.4.2 For Constructors of Verified Classes
- 7 Templates
- 8 Discussion
 - 8.1 Alternatives to Profile Suppression Annotation

- [8.2 Non-Constructor Delegation](#)
- [8.3 Divergences from \[P3274R0\]](#)
- [8.4 Overlap with \[P3081R2\]](#)
- [8.5 Overlap with \[P1179R1\]](#)
- [8.6 Inconsistency Between Declarations and Implementations](#)
- [9 Wording](#)
- [10 Future Work](#)
- [11 Conclusion](#)
- [12 Acknowledgements](#)
- [13 Straw Polls Concluded](#)
- [14 Straw Polls To Consider](#)
- [15 Revision History](#)
- [16 References](#)

§ 1 Abstract

We propose the `std::initialization` safety profile. Once enforced, this profile ensures initialization of variables to determinate values, under a limited set of assumptions. This profile’s sole objective is to prevent undefined or erroneous behavior related to a lack of initialization. This safety profile prohibits some C++ features, and restricts constructors. Existing code bases are likely to violate these constraints, and thus this feature is an opt-in via an attribute.

§ 2 Introduction

There is a growing push towards greater memory safety and memory-safe languages. While C++ is not memory-safe, it is desirable to specify an opt-in mechanism allowing a subset of C++ features that would result in memory safe programs. This has been termed ‘profiles’ ([P3274R0]), and would be specified at the translation unit level using an attribute.

In this paper, we propose the `initialization` profile, which operates at the ‘enforce’ level ([P3081R2], [P3589R1]), and provides guarantees about variables’ initialization.

The examples in this paper assume that the profile is enabled at the ‘enforce’ level, unless annotated otherwise.

§ 2.1 Industry Demand

Industry compliance standards, such as CERT C++ [CERT], forbid access of uninitialized memory (Rule EXP53-CPP). While they imply complete initialization, they do not specify how to achieve that objective.

However, the automobile safety industry is a bit more specific. For instance, the MISRA C++ standard [MISRA], specifically advise proper initialization of class objects.

MISRA C++2023 Rule 15.1.2 “All constructors of class should explicitly initialize all of its virtual base classes and immediate base classes”

MISRA C++2023 Rule 15.1.4 “All direct, non-static data members of a class should be initialized before the class object is accessible”

§ 3 Design Objectives

§ 3.1 Main Objective

The main objective of this profile is to eliminate the risk of undefined/erroneous behavior due to uninitialized memory, with the following assumptions:

- Inputs to verified code are initialized to determinate values.
- There are no dangling pointers or references (i.e. the `std::lifetime` profile is enforced).
- There is no `constexpr` stripping (i.e. the `std::type` profile is enforced).
- The programmer uses the exemption mechanisms in a manner that’s memory-safe.
- The program is well-formed.

§ 3.2 Profile Quality Objectives

In addition, we have following non-functional objectives:

Simplicity of specification and use:

- the set of rules should be small, with few exceptions.
- minimal or no annotations are needed

Simplicity of verification:

- Verification should be done by intraprocedural analysis.
- Verification should not require dataflow analysis.

Industrial applicability:

- This profile should be useable on older code bases in an incremental manner.

- The profile should rely on C++11 language features, except for any attribute specified in a later version of C++.
- Verified code should be able to interact with non-verified code in a safe manner.
- No cascading effect: a construct being profile-rejected should not affect other program constructs.

§ 3.3 Non-objectives

This profile does not address the overrun problem.

§ 4 Definitions

A *verified global* is a variable with static storage duration (sec 6.7.6.2 [\[basic.stc.static\]](#)) or thread storage duration (sec 6.7.6.3 [\[basic.stc.thread\]](#)) on which the profile is enforced.

A *verified class* is a class that on which the profile is enforced.

A *verified function* is a function on which the profile is enforced. This includes the member functions of a verified class, and lambdas defined by a verified function.

A *verified data member* is a non-static data member of a verified class that is not exempted from verification.

An *object parameter* is either the `this` pointer or an explicit object parameter (sec 9.3.4.6 [\[dcl.fct\]](#)).

A *verified variable* is any of the following * verified globals * verified data member * variable with automatic storage duration in a verified function. This includes object parameters of verified functions. * formal parameter of a verified function

Non-verified code is code for which the `std::initialization` profile is not enforced.

Acceptable inputs are:

1. The non-exempt transitive closure of reachable verified variables
2. The result applying the pointer dereference (`*`), address-of (`&`) built-in operators, or an implicit conversion to boolean, on the symbols obtained from 1.
3. Manifestly constant-evaluated expressions (sec 7.7 [\[expr.const\]](#))
4. Arithmetic operations whose operands are the above (except pointer arithmetic)
5. Return values of verified functions when the arguments of the function call are all acceptable inputs.

The *non-exempt transitive closure* of X means the set of symbols that are reachable from X using built-in the dot (`.`) and arrow (`->`) operators, except for symbols exempt from

verification.

For instance:

```
struct S {
    int b;
    OtherStruct* c;
    int * exempted [[profiles::suppress(std::initialization)]];
    int * non_exempted;
    OtherStruct exempted_struct [[profiles::suppress(std::initialization)]];
};

void verified_function(S &s) {
    auto a = s; //acceptable
    auto b = s.b; //acceptable
    auto c = s.c->c2(); //acceptable if c2 is a
        verified function
    auto d = s.exempted; //not acceptable
    auto e = s.non_exempted[32]; //not acceptable
    [[profiles::suppress(std::initialization)]] {
        auto f = s.non_exempted[32]; //acceptable
    }
    auto g = *s.non_exempted; //acceptable
    auto h = &s.b; //acceptable
    auto i = s.exempted_struct.c1; //not acceptable
    auto j = *s.c; //acceptable
    bool k = s.b; //acceptable
    auto l = &(s.c->c1); //acceptable
    auto local_exempt [[indeterminate]] = s.b; //not acceptable
    auto m = local_exempt; //not acceptable
    auto n = *(s.non_exempted + 32); //not acceptable
}
```

§ 5 Exemptions and Interactions with Non-Verified Code

Developers can exempt variables from verification using the `[[indeterminate]]` attribute from [P2795R5] or `[[profiles::suppress(...)]]` attribute from [P3081R2].

```
struct HighPerformance {
    std::byte* buf [[profiles::suppress(std::initialization)]];
    int sz = -1;
    void fill(/*...*/);
};

void verified_fn(const HighPerformance & hp) {
    std::byte *buf [[indeterminate]] = hp.buf;
}
```

In addition, developers can suppress profile enforcement for code regions using the `[[profiles::suppress(...)]]` attribute from [P3081R2].

```
[[profiles::suppress(std::initialization)]]
int non_verified(int a);

void suppress_example(const S *s) {
    [[profiles::suppress(std::initialization)]]{
        auto v = s->non_exempted[32];
        do_something(v);
    }
}
```

Note that verified code can call non-verified functions, with some limitations. There are no limitations on non-verified code using verified code.

§ 6 Constraints on Verified Code

The following constraints must be satisfied by all code under the purview of that profile, except

- Program constructs not subject to being profile-rejected (as mentioned in `[dcl.attr.profile]` in [P3081R2](#)).
- Program constructs subject to being profile-rejected that are not potentially-evaluated (sec 7.2.3 [\[expr.context\]](#)).

Otherwise, the translation unit is profile-rejected.

§ 6.1 General Constraints

For all verified variables the following constraints apply:

- `general.always.init` Default initialization that result in no initialization (sec 9.4 [\[dcl.init\]](#)) is prohibited. This rule also applies to arrays and dynamically-allocated arrays (sec 7.6.2.8 [\[expr.new\]](#)).
- `general.verif.init` Verified variables can only be initialized by default initialization or acceptable inputs. Likewise, when verified variables are the target of assignments, the assigned value must be a verified input.
- `general.type` Verified variables' type can only be trivial and verified classes, or pointers or references or arrays thereof.

Examples:

```
struct pod {
    int i;
    int j;
};
struct DefaultDoesNotInitialize {
    pod p;
    DefaultDoesNotInitialize() = default;    //profile-rejected:
    general.always.init
};
```

```

pod podFactory() {
    pod p;    // profile-rejected: general.always.init
    return p;
}

struct InitsWithNonVerified {
    int _i;
    int _j;
    InitsWithNonVerified(int &i) : _i(i), _j(non_verified_function())
        //profile-rejected: general.verif.init
    {}
};

void UnsafeUpdateArg(pod& p) {
    p.i = non_verified_function(); //profile-rejected: general.verif.init
}

class [[profiles::suppress(std::initialization)]] NonVerifiedClass { /**/
};

void non_verified_in(const NonVerifiedClass &uc) { //profile-rejected:
    general.type
    //...
}

struct parent1 {
    int i;
    parent1() = default; //profile-rejected: general.always.init
};
struct child1 : public parent1 {
    int j;
    child1() : parent1(), j(42) {} //child is compliant, but parent isn't
}

```

§ 6.2 Verified Globals

Variables with either static storage duration (sec 6.7.6.2 [\[basic.stc.static\]](#) - including static data members (sec 11.4.9.3 [\[class.static.data\]](#)) in a verified class) or thread storage duration (sec 6.7.6.3 [\[basic.stc.thread\]](#)) are guaranteed to be initialized with constant initialization (sec 6.9.3.2 [\[basic.start.static\]](#)). However, they can be reassigned with dynamic initialization (sec 6.9.3.3 [\[basic.start.dynamic\]](#)).

Dynamic initialization can lead to subtle bugs, such as:

- Circular dependencies
- Use of statically-initialized values before they are dynamically-initialized.
- Initialization order issues when dealing with variables defined in other translation units.
- Assignment of uninitialized values

We illustrate how uninitialized memory can affect static data members with dynamic initialization below.

```
struct GetsCorrupted {
    GetsCorrupted() : thefield(0) {} //compliant
    int thefield;
};

struct Wrapper {
    Wrapper() = default; //Not a POD
    static GetsCorrupted wrapped;
};

[[profiles::suppress(std::initialization)]]
GetsCorrupted corruptingFactory() {
    GetsCorrupted ret{}; //All initialized, good
    ret.thefield = non_verified_fn(); //Now, some uninitialized memory
    snuck in
    return ret;
}
GetsCorrupted Wrapper::wrapped = corruptingFactory(); //profile-rejected:
global.static.init, general.verif.init
```

The use of verified functions improve the picture, but initialization order issues remain:

```
extern int externInt;
int readsExtern() {
    return externInt; //compliant
}
int globalIntInitWithVerifFunc = readsExtern(); //profile-rejected:
global.static.init
```

For all verified globals, the following constraint applies:

- `global.static.init` Verified globals can only be initialized using constant or zero initialization.

§ 6.3 Verified Classes

All verified classes must satisfy the following property:

- `base.are.verified` All base classes of verified classes must be verified classes.
- `derived.are.verified` All derived classes of verified classes must be verified classes.

Examples:

```
//Not a verified class, but would be compliant if it were
struct [[profiles::suppress(std::initialization)]] NonVerifiedBaseClass {
    int i = 0;
    NonVerifiedBaseClass() = default;
};
struct VerifiedDerivedClass : public NonVerifiedBaseClass {
    int j;
```



```

VerifiedDerivedClass() : NonVerifiedBaseClass(), j(42) {} //profile-
    rejected: base.are.verified
};

struct [[profiles::suppress(std::initialization)]]
    NonVerifiedDerivedClass : public VerifiedDerivedClass {
    int k;
    NonVerifiedDerivedClass() : VerifiedDerivedClass(), k(42) {} //profile-
        rejected: derived.are.verified
};

```

§ 6.4 Verified Functions

§ 6.4.1 For All Verified Functions

All verified functions must also satisfy the following properties:

- `restrict`.returns The function can only return an acceptable input, except for
 - a reference or a pointer to a local variable
 - lambdas defined in the function
 - local variables of any `std::function` template instantiation
- `no.ref.args` Acceptable inputs can be passed to functions as follows:
 - Verified functions: copy, reference, pointer
 - Non-verified functions: copy, `const` reference, `const` pointer.
- `verified.overrides` When a member function `overrides` a `virtual` verified function, the overriding function must also be a verified function.

Examples:

```

void SafeUpdateArg(pod& p) {
    p.i = verified_function(); //Compliant
}

int verified_uses_unverified_compliant(int i) {
    int tmp = 0;
    [[profiles::suppress(std::initialization)]] {
        tmp = non_verified_function(); //Compliant
    }
    return i * tmp;
}

[[profiles::suppress(std::initialization)]]
void non_verified_function(int& mutate);

struct CallsNonVerifiedWithReference {
    int _i;
    int _j;
    CallsNonVerifiedWithReference(int &i) : _i(i), _j{} {
        non_verified_function(i); //profile-rejected: no.ref.args
    }
};

```

```

    }
};

class [[profiles::suppress(std::initialization)]] NonVerifiedBase {
public:
    NonVerifiedBase() = default;
protected:
    virtual void [[profiles::enforce(std::initialization)]]
        verified_protected_fn();
private:
    // ...
};

class [[profiles::suppress(std::initialization)]] NonVerifiedDerived :
    public NonVerifiedBase {
public:
    NonVerifiedDerived() = default;
protected:
    void verified_protected_fn() override; //profile-rejected:
        verified.overrides
};

class VerifiedBase {
public:
    VerifiedBase() = default;
protected:
    virtual void verified_protected_fn();
private:
    // ...
};

class VerifiedDerived : public VerifiedBase {
public:
    VerifiedDerived() = default;
protected:
    void [[profiles::suppress(std::initialization)]]
        verified_protected_fn() override; //profile-rejected:
        verified.overrides
};

auto unsafeRetLambda(int i){ //passed by copy
    auto lambda = [&i]() { //reference to local variable
        return i * i;
    };
    return lambda; //profile-rejected: restrict.returns
}

```

§ 6.4.2 For Constructors of Verified Classes

In addition to the properties that apply to verified functions, all constructors of a verified class must satisfy the following properties:

- `init.before.read` All verified data members must be initialized before being read.

- `init.all` Except for delegating constructors, the constructor must initialize all verified data members.
- `init.list` When verified data members must be initialized, the constructor must use the *mem-initializer-list* (sec 11.9.3 [\[class.base.init\]](#)) to initialize them.

Note regarding `init.list`: The following data members are exempt from the `init.list` criteria:

- is declared with a *brace-or-equal-initializer* (sec 11.4.1 [\[class.mem.general\]](#))
- is of a type with a default constructor and its omission from the *mem-initializer-list* would result in the use of that default constructor

A data member is considered read whenever it is present in the function, except when:

- It is covered by the exceptions covered above.
- It is the left operand of a simple assignment (sec 7.6.19 [\[expr.ass\]](#)).

§ 6.4.2.1 Compliant Examples

```
struct ValueInitialized {
    pod p{};
    ValueInitialized() = default; //compliant
};

struct InitWithVerifiedReturnValue {
    static pod podFactory();
    InitWithVerifiedReturnValue() : p(podFactory()) {} //compliant
};

struct WithExemption {
    std::byte* buf [[indeterminate]];
    size_t    buf_size;
    int i;
    WithExemption() : buf_size(0), i(0) {} //compliant: buf is not a
        verified data member
};

struct SafeDefaultInit {
    int i;
    int j;
    SafeDefaultInit() : i(123), j(456) {} //compliant
};

struct ReliesOnDefaultInit {
    int i;
    SafeDefaultInit sdi;
    ReliesOnDefaultInit() : i(123) {} //compliant: sdi has a default ctor
};

struct MixedInits {
    int i;
    int j;
```

```

int z = 0;
MixedInits() : i(123), j(456) {} //compliant: verified data members are
                               initialized using either allowed mechanism
};

struct WithCallInCtorBody {
    int i;
    int j;
    void utility_function() const;
    WithCallInCtorBody(int i) : i(i), j() {
        utility_function(); //compliant: calling a verified function with
                             'this'
    }
};

struct UpdatesGlobal {
    static unsigned num_allocations;
    UpdatesGlobal() {
        ++num_allocations; //compliant: a verified input is updated with the
                             result of an arithmetic operation over verified inputs
    }
};
unsigned UpdatesGlobal::num_allocations = 0;

struct CallsVerifiedNonConst {
    int i;
    int j;
    void mutating();
    CallsVerifiedNonConst(int i) : i(i), j{} {
        mutating(); //compliant, but could have a redefinition of i or j
    }
};

```

§ 6.4.2.2 Profile-Rejected Examples

```

struct WrongOrder {
    int i;
    int j;
    WrongOrder() : i(j), j(42) {} //profile-rejected: init.before.read
};

struct MissingInit {
    int i;
    MissingInit() {} //profile-rejected: init.all
};

struct InitInCtorBody {
    int i;
    int j;
    int z = 0;
    InitInCtorBody() {
        i = 123;
        j = 456;
        //profile-rejected: init.list
    }
};

```

```
struct CallsNonVerifiedWithFieldReference {
    int _i;
    int _j;
    CallsNonVerifiedWithFieldReference(int &i) : _i(i), _j{} {
        non_verified_function(_j); //profile-rejected: no.ref.args
    }
};
```

§ 7 Templates

In the case of templated classes, the property is verified during template instantiation.

```
class [[profiles::suppress(std::initialization)]] Suppressed {/**/};
class Enforced {/**/};

template<typename T>
class Template {
    T field = T();
};

void uses_template() {
    Template<Suppressed> sup {}; //profile-rejected: general.type
    Template<Enforced> enf {}; //compliant
}
```

§ 8 Discussion

§ 8.1 Alternatives to Profile Suppression Annotation

An earlier version of this paper proposed `std::verified_cast` as a terser alternative to block-level disablement and included it in the profile specification. This function would allow programmers to use return values from non-verified functions in initialization lists in a way that’s more natural than profile disablement. It would be the C++ equivalent of Rust’s `unsafe` expression.

However, it would be desirable to add a template parameter allowing programmers to specify which profile(s) are suppressed (e.g. `std::verified_cast<std::initialization>`), and allow for a full equivalency to Rust’s `unsafe` expression with `std::verified_cast<std::all>`. However, what exactly should be the template type of `std::verified_cast<>`? `std::all` and `std::initialization` are not types, but strings that a compiler would recognize in a profile attribute. Allowing `std::verified_cast<>` would thus require language-level changes, or the definition of types in the standard library that have little meaning outside of safety profiles. Both options are likely to reduce consensus.

```
class ForVCast {
    int i;
    int j;
```

```

int k;
ForVCast(): i(std::verified_cast<std::initialization>(non_verified())),
             j([[profiles::suppress(std::initialization)]] {
non_verified() } ), //doesn't compile
             k([]() [[profiles::suppress(std::initialization)]] { return
non_verified(); }())
             {}
}

```

During the Hagenberg meeting, SG23 attendees brought up the similarities between `std::verified_cast` and `std::launder`. We consider that `std::launder` is not a good way to suppress profile enforcement, as this would embue existing code with additional properties that its authors did not anticipate.

This proposal is ultimately orthogonal to the profile specification and, given the technical constraints, we would pursue it in a separate paper going forward.

§ 8.2 Non-Constructor Delegation

A recent SG23 mailing list discussion highlighted that delegating initialization to a non-constructor member function is idiomatic in C++. Supporting this idiom would make this profile more useful.

The bit of code that triggered the discussion is the following:

```

basic_string(const _CharT* __s, const _Alloc& __a = _Alloc())
: _M_dataplus(_M_local_data(), __a)
{
    //...
    _M_construct(__s, __end, forward_iterator_tag());
}

```

In this case, `_M_local_data()` returns a `const` pointer to a data member (`_M_local_buf`) and passes it to the `_M_dataplus` data member. The initialization then is done by `_M_construct`. This code would be reported as violating the safety profile as we specify it in this draft, since the constructor does not initialize `_M_dataplus` directly.

There are a few solutions to this problem:

1. Force developers to rewrite their code to use delegating constructors.
2. Use an on-demand interprocedural analysis that ensures that initialization happens on all paths in the callee.
3. A suggestion was to annotate arguments that must be initialized with `[[must_init]]`.

```

struct DelegatingInit {
    int member;
    DelegatingInit() {
        internal_init(&member);
    }
}

```

```
void internal_init([[must_init]] int* p);  
}
```

The latter option is less intrusive than option 1, and would be simpler to verify than option 2, simply because the scope of the analysis becomes well-bounded. As such, it is worth considering.

Nonetheless, we consider it undesirable for the following reasons:

- Adding another attribute is undesirable.
- Only explicit parameters can be specified this way. However, developers might like to have a general `initialize()` member function.
- Functions with the `[[must_init]]` attribute can delegate further to other `[[must_init]]` functions, possibly leading to recursion.
- Verifying such an annotated function would require dataflow analysis.

Given the design objectives, we conclude that there is no viable path to accept non-constructor delegation in this profile.

§ 8.3 Divergences from [P3274R0]

This draft materially deviates from [P3274R0] in the following ways:

- We do not assume that all default constructors are initializing data members.
- We do not take position on implicit initialization and whether it is error-prone.

§ 8.4 Overlap with [P3081R2]

The rule for Type.6 proposed by [P3081R2] is identical to rule `general.always.init`. Since this rule is not related to type safety, it belongs more meaningfully to the initialization profile.

Since this restriction is very desirable, and that [P3081R2] is foundational to this paper, we encourage [P3081R2] to specify an initialization profile containing only rule Type.6, which this paper would eventually build upon.

§ 8.5 Overlap with [P1179R1]

The rule `restrict.returns` overlaps with the `std::lifetime` profile to some extent ([P1179R1]), as it aims to avoid a read of nondeterminate memory contents by the caller of a verified function. Thus, the rule leads to profile-rejected safe code, as illustrated below.

```
auto safeRetLambda(int i){ //passed by copy  
    auto lambda = [i]() { // copy of local variable  
        return i * i;  
    };  
};
```

```
return lambda; //safe, but profile-rejected: restrict.returns
}
```

It would be desirable to ensure that only function objects capturing local variables be prohibited. But that seems difficult to do so without dataflow analysis. In the example below, only `lambda1` would be unsafe.

```
std::function<int()> unsafeStdFunction(int i){ //passed by copy
    std::function<int()> lambda1 = [&i]() { // reference to local variable
        return i * i;
    };
    std::function<int()> lambda2 = [i]() { // copy of local variable
        return i * i;
    };
    return nondet()? lambda1 : lambda2; //profile-rejected: restrict.returns
}
```

It may be preferable to delegate cases like these to the `std::lifetime` profile and rewrite the rule as follows:

— `restrict.returns`: The function can only return an acceptable input.

§ 8.6 Inconsistency Between Declarations and Implementations

Different translation units may have different profiles enabled. This could lead to situations where a translation unit mistakenly expects a symbol to comply with the requirements of this profile. Consider the example below, whereby the translation unit implementing `foo` does so without the initialization profile. However, another translation unit requires the initialization profile, and depends on `foo`.

```
//in impl.cpp
[[profiles::suppress(std::initialization)]]
int foo() { /*...*/ }

//in caller.cpp
[[profiles::enforce(std::all)]]
int foo();

int main(int argc, char** argv) {
    return foo();
}
```

This issue was anticipated in part in the wording of [P3081R2], section [dcl.attr.profile]. We consider that this situation make the program ill-formed, no diagnostic required (IFNDR). This is line with other attributes in the standard (sec 9.12 [dcl.attr]) and should be added to the general profile specification.

§ 9 Wording

We add a paragraph to section 6.9.3.2 [\[basic.start.static\]](#) as follows:

- x. Whenever the `std::initialization` profile is enforced, any dynamic initialization must be performed as static initialization under the terms of paragraph 3. Otherwise, the statement is profile-rejected by `std::initialization`.

We add a paragraph to section 9.4 [\[dcl.init\]](#) as follows:

- x. Initialization-verified variables, except for function formal parameters, must either be default-initialized, or be initialized using only initialization-acceptable inputs. Otherwise, the class declaration is profile-rejected by `std::initialization`.

We add a paragraph to section 11.7.2 [\[class.mi\]](#) as follows:

- x. All base classes of an initialization-verified class are also initialization-verified classes. Otherwise, the class declaration is profile-rejected by `std::initialization`.
- y. Whenever any base classes is an initialization-verified class, the derived class is also initialization-verified. Otherwise, the class declaration is profile-rejected by `std::initialization`.

We add a paragraph to section 11.7.3 [\[class.virtual\]](#) as follows:

- x. A function overriding a virtual initialization-verified function must also be initialization-verified. Otherwise, the function declaration is profile-rejected by `std::initialization`.

We add a section to section 8.7.4 [\[stmt.return\]](#) as follows:

- x. The return statement of an initialization-verified function must return an initialization-acceptable input, except a lambda defined in the function or any local variable instantiating the `std::function` template.

We add a paragraph to section 7.6.1.3 [\[expr.call\]](#) as follows:

- x. Arguments to function calls which are initialization-acceptable inputs, within an initialization-verified function, for which call targets are not initialization-verified must only be passed by copy, constant reference or constant pointer. Otherwise, the expression is profile-rejected by the `std::initialization` profile.

We modify section 11.9 [\[class.init\]](#) as follows:

- 16 Member functions (including virtual member functions, [\[class.virtual\]](#)) can be called for an object under construction. Similarly, an object under construction can be the operand of the typeid operator ([\[expr.typeid\]](#)) or of a `dynamic_cast` ([\[expr.dynamic.cast\]](#)). However, if these operations are performed in a ctor-initializer (or in a function called directly or indirectly from a ctor-initializer) before all the mem-initializers for base classes have

completed, the program has undefined behavior if the `std::initialization` profile is not enforced. Otherwise, it is profile-rejected by the `std::initialization` profile.

- x. Non-delegating constructors that are initialization-verified functions must initialize all initialization-verified non-static data members declared in their class in a *mem-initializer-list*, except for data members declared with a brace-or-equal-initializer (sec 11.4.1 11.4.1 [class.mem.general]) and data members of a type with a default constructor and its omission from the *mem-initializer-list* would result in the use of that default constructor. Otherwise, the constructor is profile-rejected by the `std::initialization` profile.
- y. Non-delegating constructors that are initialization-verified functions must initialize initialization-verified non-static data members declared in their class before they are read. Otherwise, the constructor is profile-rejected by the `std::initialization` profile.

We add a section to [expr.assign] as follows:

- x. When the left operand of a simple assignment is an initialization-verified variable, the right operand must be an initialization-acceptable input.

The following wording changes section assumes that [P3081R2] and [P3589R1] are adopted. We propose insertions into the former’s wording.

We add the following entry to [tab:profiles.summary]:

profile-name	Subclauses
<u>initialization</u>	6.9.3.2 [basic.start.static], 11.7.2 [class.mi], 8.7.4 [stmt.return], 7.6.1.3 [expr.call], 11.9 [class.init], 11.7.3 [class.virtual]

We add a paragraph to section [dcl.attr.profile] as follows:

- x. Whenever profile enforcement on a program construct C is not identical between translation units, the program is ill-defined, no diagnostic required.

We add a section after [dcl.attr.profile] as follows:

- x. Initialization profile
 1. Whenever the `std::initialization` profile is enforced on program construct C , the `std::type` and `std::lifetime` profiles are also enforced on C. Otherwise, the translation unit is profile-rejected by `std::initialization`.
 2. Definitions An *initialization-verified class* is a class for which the `std::initialization` profile is enforced.

An *initialization-verified function* is a function for which the `std::initialization` profile is enforced. This includes the member functions of initialization-verified classes,

and lambdas defined by initialization-verified function.

An *initialization-verified variable* is any of the following, when the `std::initialization` profile is enforced:

- a variable with static storage duration (6.7.6.2 [basic.stc.static]) or thread storage duration (6.7.6.3 [basic.stc.thread])
- a non-static data member of a initialization-verified class,
- a variable with automatic storage duration in an initialization-verified function,
- an initialization-verified function’s formal parameters.

An *object parameter* is either the `this` pointer or an explicit object parameter (9.3.4.6 [dcl.fct]).

Initialization-acceptable inputs are any of the following.

- a. The non-exempt transitive closure of reachable initialization-verified variables
- b. The result applying the pointer dereference (*), address-of (&) built-in operators, or an implicit conversion to boolean, on the symbols obtained from a.
- c. Manifestly constant-evaluated expressions (7.7 [expr.const])
- d. Arithmetic operations whose operands are the above (except pointer arithmetic)
- e. Return values of verified functions when the arguments of the function call are all acceptable inputs.

The *non-exempt transitive closure* of X means the set of symbols that are reachable from X using built-in the dot and arrow operators, and only include symbols for which the `std::initialization` profile is enforced.

3. All initialization-verified variables’ type can only be trivial types and verified classes, pointers thereof, references thereof or arrays thereof.

§ 10 Future Work

We also observe that profiles imply a constraint on what types can be used in a template. This hints at a new concept. A future revision of this paper would explore this further.

§ 11 Conclusion

We propose the `std::initialization` safety profile. When enforce, it guarantees that all affected code will initialize both local and global variables to determinate values, assuming that

- a. the data used for construction is itself initialized properly,

- b. the `std::type` and `std::lifetime` profiles are enforced on verified code and its transitive callees,
- c. the program is well-formed,
- d. code in the regions where the profile maintains the profile's properties

The profile does not depend on the presence of specific modern C++ features and can thus be applied to legacy code bases. Thus, developers on legacy code bases that are still using older versions of the C++ standard could take advantage of this profile.

§ 12 Acknowledgements

The authors would like to thank the attendees of SG23 meeting, as well as contributors on the SG23 reflector for their valuable input. We especially want to thank Tom Honermann and Jens Maurer.

§ 13 Straw Polls Concluded

The following straw polls were conducted during the Wrocław 2024 meeting.

POLL: We should promise more SG23 committee time to pursuing this paper, knowing that our time is scarce and this will leave less time for other work.

Favor	Neutral	Against
18	1	0

Strong consensus

POLL: For a given scope of applicability (eg translation unit) should this profile prohibit the use of default initialization altogether?

Favor	Neutral	Against
1	4	13

Strong consensus against

POLL: For a given scope of applicability (eg translation unit) should this profile prohibit the use of default initialization leading to no initialization?

Favor	Neutral	Against
17	0	0

Unanimous

The following straw polls were conducted during the Hagenberg 2025 meeting.

POLL: We support adding `verify_cast<>` to allow suppression of the initialization profile for specific accesses.

Favor	Neutral	Against
3	4	0

Consensus in favour

POLL: Should the initialization profile allow re-initialization in the construction body?

Favor	Neutral	Against
7	1	0

Consensus in favor

POLL: Should we allow passing verified data by non-const reference to unverified functions under this profile?

Favor	Neutral	Against
0	2	4

POLL: Should we allow passing verified data by const reference to unverified functions under this profile?

Favor	Neutral	Against
5	2	0

§ 14 Straw Polls To Consider

Q1: SG23 forwards this paper to EWG.

Q2: Should the profile rely on an attribute on parameters that indicates what the function is responsible for initializing?

Q3: The `std::lifetime` profile should prohibit to identify dangling references after function return rather than the `std::initialization` profile.

§ 15 Revision History

R3: Update for Sofia

- Removed the `no.reassign` rule based on straw poll results.
- Reduced `std::verified_cast` to a discussion, taking in consideration straw poll results.
- Addressed the discrepancy problem as IFNDR.

- Expanded the set of acceptable inputs.
- Modified the `restrict`.returns rule.
- Updated text to use a trivial type instead of a POD type.
- Added wording.

R2: 2025-01-13 Initialization at large, for Hagenberg

R1: 2024-10-11 Class initialization, presented in Wrocław

R0: 2024-09-17 Early draft on class initialization for discussion with the community

§ 16 References

[CERT] SEI CERT. 2016. SEI CERT C++ Coding Standard.

<https://wiki.sei.cmu.edu/confluence/pages/viewpage.action?pageId=88046682>

[MISRA] The MISRA Consortium and Chris Tapp. 2023. MISRA C++:2023: Guidelines for the use of C++17 in critical systems.

<https://misra.org.uk/misra-cpp2023-released-including-hardcopy/>

[P1179R1] Herb Sutter. 2019-11-22. Lifetime safety: Preventing common dangling.

<https://wg21.link/p1179r1>

[P2795R5] Thomas Köppe. 2024-03-22. Erroneous behaviour for uninitialized reads.

<https://wg21.link/p2795r5>

[P3081R2] Herb Sutter. 2025-02-04. Core safety profiles for C++26.

<https://isocpp.org/files/papers/P3081R2.pdf>

[P3274R0] Bjarne Stroustrup. 2024-05-10. A framework for Profiles development.

<https://wg21.link/p3274r0>

[P3589R1] Gabriel Dos Reis. 2025-02-02. C++ Profiles: The Framework.